

Car Project

Analysis Document

VESELIN ATANASOV | ABDULLAH ADIL ÖNCÜ

Document History

<i>V</i>	<i>DATE</i>	<i>STATUS</i>	<i>AUTHOR</i>	<i>DESCRIPTION</i>	<i>UPDATE</i>
1.0	12/6/2024	Draft	Adil	Document Creation	Document Creation
1.1	12/6/2024	Draft	Veselin, Adil	Made Introduction and Project Goal and also started working on use cases and requirements	Introduction, Project Goal, Use Cases, Requirements

Table of Contents

Document History.....	2
Introduction	4
Project Goal	4
Use Cases/Scenarios.....	4
Requirements.....	9
Must-have:.....	9
Nice-to-Have Features:.....	9

Introduction

The Car Project is developed by a 2 student group. The guidelines and requirements for this project are in line with those required for the Communications Project in semester three of the Technology profile from the course ICT at Fontys University of Applied Sciences. This document describes the use cases, the requirements and generally the analysis of this project and its goals.

Project Goal

The goal of the project is to create a CAN network which simulates the way ECU's in modern cars communicate. The car simulation system needs to have at least 4 nodes and a remote application to show statistics about the car and to control certain aspects of it.

The goal we have set for ourselves is to create a robust car communication system where different nodes interact with each other and there is a dashboard node, which serves as a centralized point for control and information to the user. There is also a remote application which shares most of the functionality of the dashboard. The dashboard node has responsibility for showing engine speed, high engine temperature alerts, low tire pressure alerts and alerts for a node that fails. The engine and tire nodes will be responsible for providing this information to the dashboard and the remote control application while they are also responsible for changing the value of their values(engine speed, engine temperature and tire pressure). The lights node is responsible for controlling the lights based on messages received from the dashboard or the remote control application.

In the final version each node should follow the functionality listed above and communication should work as intended.

Use Cases/Scenarios

Element	Detail
<i>Name</i>	Toggling Headlights
<i>ID</i>	UC_001
<i>Description</i>	The User switches the headlights either on or off.
<i>Actor(s)</i>	User, Dashboard Node, Remote Control Node
<i>Organization</i>	Not being able to toggle headlights is a big safety concern for the User and the traffic and it is illegal to not have them while driving.
<i>Benefits</i>	
<i>Frequency of Use</i>	Average frequency depending on the time period which its used.
<i>Triggers</i>	User switching the Headlights via the Dashboard or the Remote Control app.
<i>Pre-Conditions</i>	HeadLights = ON/Headlights = OFF
<i>Post-Conditions</i>	HeadLights = ON/Headlights = OFF
<i>Main Course</i>	1. User changes the headlight state through the Dashboard or a Remote Control app.

CAR PROJECT ANALYSIS DOCUMENT

	2. New headlight state is sent to CAN bus. 3. Lights Node read the message and change its state.
<i>Alternate Course</i>	-
<i>Exception Course</i>	EX ₁ : Can't communicate with Lights Node. 1. User changes the headlight state through the Dashboard. 2. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	Toggling Indicators
<i>ID</i>	UC_002
<i>Description</i>	The User switches the indicators either on or off.
<i>Actor(s)</i>	User, Dashboard Node, Remote Control Node
<i>Organization</i>	This functionality is a must have if the car itself is going into traffic since it is the law to have functional indicators and it is not safe to drive without this functionality.
<i>Benefits</i>	
<i>Frequency of Use</i>	High frequency
<i>Triggers</i>	User switching the Indicators via the Dashboard or the Remote Control app.
<i>Pre-Conditions</i>	Indicators = ON/Indicators = OFF
<i>Post-Conditions</i>	Indicators = ON/ Indicators = OFF
<i>Main Course</i>	1. User changes the headlight state through the Dashboard or a Remote Control app. 2. New Indicator state is sent to CAN bus. 3. Lights Node reads the message and changes its state.
<i>Alternate Course</i>	-
<i>Exception Course</i>	EX ₁ : Can't communicate with Lights Node. 3. User changes the indicators state through the Dashboard. 4. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	Engine RPM control
<i>ID</i>	UC_003
<i>Description</i>	The User changes the RPM of the engine via a Dashboard or a Remote Control app.
<i>Actor(s)</i>	User, Dashboard Node, Remote Control Node
<i>Organization</i>	This functionality is the backbone for the movement of a car and without it the car cannot reach certain required speeds.
<i>Benefits</i>	
<i>Frequency of Use</i>	High frequency
<i>Triggers</i>	User changes the RPM value via the Dashboard or the Remote Control app.
<i>Pre-Conditions</i>	Engine RPM value = X
<i>Post-Conditions</i>	Engine RPM value = X
<i>Main Course</i>	1. The user changes the RPM via the Dashboard or the Remote Control app. 2. New RPM value is sent to the CAN bus. 3. The Engine Node reads the new RPM value from the CAN bus. 4. The Engine Node updates its RPM value.
<i>Alternate Course</i>	-

CAR PROJECT ANALYSIS DOCUMENT

<i>Exception Course</i>	EX ₁ : Can't communicate with Engine Node. 1. User changes the RPM value through the Dashboard. 2. Because of broken communication nothing changes.
-------------------------	--

Element	Detail
<i>Name</i>	Braking
<i>ID</i>	UC_004
<i>Description</i>	The User brakes through the Dashboard and the engine stops.
<i>Actor(s)</i>	User, Dashboard Node, Engine Node
<i>Organization</i>	This functionality is a must have if the car itself is going into traffic since it is illegal to go on traffic without working brakes and it is also very dangerous.
<i>Benefits</i>	
<i>Frequency of Use</i>	High frequency
<i>Triggers</i>	User presses brakes via the Dashboard.
<i>Pre-Conditions</i>	Engine = ON
<i>Post-Conditions</i>	Engine = OFF
<i>Main Course</i>	1. The User presses the brakes through the dashboard. 2. New brake state is sent to the CAN bus. 3. The Engine Node reads the new brake state. 4. The Engine Node stops immediately.
<i>Alternate Course</i>	AC ₁ : Engine is already OFF 1. The User presses the brakes through the Dashboard. 2. New brake state is sent to the CAN bus. 3. The Engine Node reads the new brake state. 4. The Engine Node does not change anything since it is already OFF.
<i>Exception Course</i>	EX ₁ : Can't communicate with Engine Node. 1. The User presses the brakes through the Dashboard. 2. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	Temperature Reading
<i>ID</i>	UC_005
<i>Description</i>	The Engine Node will send the temperature of the engine to the Remote Control app.
<i>Actor(s)</i>	Engine Node, Remote Control Node
<i>Organization</i>	This feature will be essential for seeing the state of the engine and notice dangerous situations.
<i>Benefits</i>	
<i>Frequency of Use</i>	High Frequency
<i>Triggers</i>	Every 2 seconds
<i>Pre-Conditions</i>	Temperature = X
<i>Post-Conditions</i>	Temperature = Y
<i>Main Course</i>	1. The Engine Node will wait 2 seconds. 2. The Engine Node will read the temperature value. 3. The Engine Node will send the temperature reading to the CAN bus.
<i>Alternate Course</i>	-
<i>Exception Course</i>	EX ₁ : Can't communicate with Remote Control Node.

CAR PROJECT ANALYSIS DOCUMENT

1. The Engine Node will wait 2 seconds.
2. The Engine Node will read the temperature value.
3. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	High Temperature Warning
<i>ID</i>	UC_006
<i>Description</i>	The Engine Node will send a warning to the Dashboard and the Remote Control app if the temperature exceeds a certain value.
<i>Actor(s)</i>	Engine Node, Dashboard Node, Remote Control Node
<i>Organization</i>	This feature will be essential for seeing the state of the engine and notice dangerous situations.
<i>Benefits</i>	
<i>Frequency of Use</i>	Low Frequency
<i>Triggers</i>	Temperature > DangerousTemperature
<i>Pre-Conditions</i>	Warning = false
<i>Post-Conditions</i>	Warning = true
<i>Main Course</i>	1. The Engine Node will wait 2 seconds. 2. The Engine Node will read the temperature value. 3. The Engine Node will compare the temperature value with DangerousTemperature value. 4. The Engine Node will send a warning message to the Dashboard and the Remote Control app.
<i>Alternate Course</i>	-
<i>Exception Course</i>	EX ₁ : Can't communicate with Dashboard or Remote Control Node. 1. The Engine Node will wait 2 seconds. 2. The Engine Node will read the temperature value. 3. The Engine Node will compare the temperature value with DangerousTemperature value. 4. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	Tire Pressure Reading
<i>ID</i>	UC_007
<i>Description</i>	The Engine Node will send the tire pressure of the tires to the Remote Control app.
<i>Actor(s)</i>	Tires Node, Remote Control Node
<i>Organization</i>	This feature will be essential for seeing the state of the tires and notice dangerous situations.
<i>Benefits</i>	
<i>Frequency of Use</i>	High Frequency
<i>Triggers</i>	Every 2 seconds
<i>Pre-Conditions</i>	Tire Pressures = {X, X, X, X}
<i>Post-Conditions</i>	Tire Pressures = {Y, Y, Y, Y}
<i>Main Course</i>	1. The Tires Node will wait 2 seconds. 2. The Tires Node will read the tire pressure values. 3. The Tires Node will send the tire pressure readings to the CAN bus.

CAR PROJECT ANALYSIS DOCUMENT

<i>Alternate Course</i>	-
<i>Exception Course</i>	EX ₁ : Can't communicate with Remote Control Node. 1. The Tires Node will wait 2 seconds. 2. The Tires Node will read the tire pressure values. 3. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	Tire Pressure Warning
<i>ID</i>	UC_008
<i>Description</i>	The Tires Node will send a warning to the Dashboard and the Remote Control app if the tire pressure is less than a certain value.
<i>Actor(s)</i>	Tires Node, Dashboard Node, Remote Control Node
<i>Organization Benefits</i>	This feature will be essential for seeing the state of the tires and notice dangerous situations.
<i>Frequency of Use</i>	Low Frequency
<i>Triggers</i>	TirePressure[x] < DangerousTirePressure
<i>Pre-Conditions</i>	Warning = false
<i>Post-Conditions</i>	Warning = true
<i>Main Course</i>	1. The Tires Node will wait 2 seconds. 2. The Tires Node will read the tire pressure values. 3. The Tires Node will compare the tire pressure values with DangerousTirePressure value. 4. The Tires Node will send a warning message to the Dashboard and the Remote Control app.
<i>Alternate Course</i>	-
<i>Exception Course</i>	EX ₁ : Can't communicate with Dashboard or Remote Control Node. 1. The Tires Node will wait 2 seconds. 2. The Tires Node will read the tire pressure values. 3. The Tires Node will compare the tire pressure values with DangerousTirePressure value. 4. Because of broken communication nothing changes.

Element	Detail
<i>Name</i>	Node Failure
<i>ID</i>	UC_009
<i>Description</i>	If one of the nodes fail this will be shown in the Dashboard and the Remote Control app.
<i>Actor(s)</i>	Lights Node, Engine Node, Tires Node, Dashboard Node, Remote Control Node
<i>Organization Benefits</i>	This feature will be essential for seeing the state of the CAN bus connections and notice dangerous situations.
<i>Frequency of Use</i>	Low Frequency
<i>Triggers</i>	Node communication failure
<i>Pre-Conditions</i>	<type>NodeState = FUNCTIONAL

<i>Post-Conditions</i>	<type>NodeState = FAILED
<i>Main Course</i>	1. The heart beat of on of the nodes stop. 2. The state of the NodeState will be changed accordingly. 3. This will be made visible in Dashboard and the Remote Control app.
<i>Alternate Course</i>	-
<i>Exception Course</i>	-

Requirements

Must-have:

1. Headlights control:
 - The headlights must be controlled via both the dashboard and a remote-control app.
2. Indicators control:
 - The indicators must be switchable from both the dashboard and remote-control app.
3. Engine speed control:
 - The speed of the engine must be controllable via both the dashboard and remote-control app.
4. Brakes and engine interaction:
 - The brakes must be controllable via the dashboard.
 - When the brakes are pressed, the engine must stop immediately.
5. Engine temperature monitoring:
 - The engine temperature must be readable via the app.
 - The engine temperature must be updated every two seconds.
 - If the engine temperature rises above a safe level, a warning must be displayed on both the app and the dashboard.
6. Tire pressure monitoring:
 - The tire pressure must be readable via the app.
 - The tire pressure must be updated every two seconds.
 - If the tire pressure drops below a safe level, a warning must be displayed on both the app and the dashboard.
7. Node failure detection:
 - If any system node fails, this failure must be indicated on both the app and the dashboard.

Nice-to-Have Features:

1. Event Data Recorder:

- The car must have an Event Data Recorder (EDR) that logs data such as speed, user activities, engine temperature, and tire pressure for the last "x" minutes.
 - An employee of a car repair shop must be able to read the contents of the EDR via the app upon request.
2. Cruise Control:
- The car should have cruise control that can be operated via both the dashboard and the app.